BAHRIA UNIVERSITY ISLAMABAD

Department of Computer Science

Assignment 1 — Project Proposal

KharchaBook — Student Expense Manager

A simple single-user expense tracker for students (Flutter + Supabase)

Subject	Mobile Application Development (MAD) — BS-IT 6
Class / Section	IT-6A
Group Member 1	Zain Kashif — Enrollment: 01-135232-107
Group Member 2	Huraira Minhas — Enrollment: 01-135232-030
Submission	As on LMS
Project Type	Single-user, client–server (Flutter app + Supabase cloud database)

This proposal covers all clauses required in the assignment brief: title, problem, objectives, users, reference, features, app flow, technical requirements, database + schema, scope/limitations, future enhancements, and approval.

Table of Contents — 1. Title & Description | 2. Problem | 3. Objectives | 4. Users | 5. Reference | 6. Features | 7. App Flow | 8. Technical | 9. Database | 10. DB Structure | 12. Scope | 13. Future | 14. Approval

1. App Title & Description

App Title: KharchaBook — Student Expense Manager. KharchaBook is a lightweight mobile app that lets a student record daily income and expenses in under 10 seconds, organize them by category (Food, Transport, Hostel, Study, Other), set a monthly budget, and see monthly totals with category-wise charts. It is intentionally single-user and simple so UI (Assignment 2) and full CRUD + auth + validation (Assignment 3) can be completed by a 2-member team within the semester.

2. Problem Statement

Students living on pocket-money / hostel budgets overspend on food, transport and daily items because cash transactions are forgotten and existing finance apps are too complex, ad-heavy, and not built for student budgets. There is no simple record of ‘where did my money go this month?’ and no early warning when the monthly budget is about to be exceeded. How Might We give Bahria students the simplest possible way to track every rupee and stay within budget?

3. Project Objectives

- **O1 — Fast recording:** add an income/expense entry with title, amount, category and date with validation and error handling.
- **O2 — Organize & retrieve:** view, edit, delete entries; search by title and filter by category / month / type.
- **O3 — Insight:** show monthly total, income vs expense, and category-wise breakdown (pie/bar) on dashboard and stats screen.

- **O4 — Budget control:** let user set a monthly budget and trigger a local notification / warning when spending exceeds 80% and 100%.
- **O5 — Secure per-user data:** Supabase Auth + row-level security so each student only sees their own data, with clean navigation and tested APK.

4. Target Users

Primary users: undergraduate students (e.g., BS-IT), especially hostel / commuting students managing monthly pocket-money. **Single role only** — no admin, no multi-role complexity. Secondary beneficiary: parents who receive a clearer monthly summary. The app assumes one phone = one student, basic Android familiarity, occasional internet (cloud sync) with graceful offline error messages.

5. Reference / Justification

This is a **client–server** app (Flutter client + Supabase cloud backend), so it directly satisfies the client-server model. As required, we also cite an existing Play Store reference and external sources:

- Reference app (simplified inspiration): **Money Manager Expense & Budget** — https://play.google.com/store/apps/details?id=com.realbyteapps.moneymanagerfree (we build a student-only, simpler subset: no bank sync, no PC edit).
- Backend reference: Supabase Auth + Postgres + supabase_flutter SDK — https://supabase.com/docs.
- UI/charts reference: Flutter Material 3 + fl_chart — https://docs.flutter.dev.
- No dataset needed; categories are seeded locally (Food, Transport, Hostel, Study, Health, Other).

6. Main Features

#	Feature	Details (maps to Assignment 3 checks)
F1	Registration / Login / Logout	Supabase email+password auth, session persist, validation, error messages
F2	Add / Update / Delete / View expenses	Full CRUD with form validation (title, amount>0, date, category)
F3	Search & filter	Search by title; filter by category, month, income/expense
F4	Dashboard + Stats	Monthly total, income vs expense, category pie/bar via fl_chart
F5	Budget + notification	Set monthly budget; local notification at 80%/100%
F6	Profile management	View/edit name + budget, change password, logout

Table 1: Feature list. All features include data validation and appropriate error handling.

7. App Flow Screens – Basic Sketch / Wireframe

Navigation flow (arrows = navigation):

Splash → Login ↔ Register → Dashboard(Home) → Add/Edit Expense → Dashboard | Dashboard → Stats → Dashboard | Dashboard → Profile/Settings → Login (on logout)

Screen	Contents	Goes to
S1 Splash	Logo + app name, 2s, session check	Login or Dashboard

Screen	Contents	Goes to
S2 Login	Email, password, error text, link to Register	Dashboard / Register
S3 Register	Name, email, password, validation	Dashboard / Login
S4 Dashboard	Month total, budget bar, expense list, search+filter, FAB Add, bottom nav	Add/Edit, Stats, Profile
S5 Add/Edit	Title, amount, type toggle, category dropdown, date picker, note, save/delete	Back to Dashboard
S6 Stats	Category pie + monthly bar, month selector	Dashboard
S7 Profile	Name, email, monthly budget, logout	Login (after logout)

Figure 1 (text wireframe): 7 screens, bottom-nav on S4/S6/S7. Assignment 2 will deliver these screens with navigation, colors, typography, spacing, icons; Assignment 3 wires them to Supabase.

8. Technical Requirements

Item	Choice
Language / Framework	Dart + Flutter (Material 3), Android SDK
IDE / Tools	VS Code + Android Studio, Flutter SDK >=3.22, Git
Backend (cloud)	Supabase: Auth (email/password), Postgres, supabase_flutter SDK
Packages	supabase_flutter, fl_chart, flutter_local_notifications, intl, shared_preferences (session/cache)
Testing / Build	Flutter analyze + manual functional testing; release APK for Assignment 3

Portability note (migrate off Supabase easily): UI never imports Supabase directly. All data access goes through *abstract ExpenseRepository / AuthRepository* (DTOs: Expense, Profile). *SupabaseExpenseRepository* is one implementation; a *SqliteExpenseRepository* (drift/sqflite) or Firebase one can replace it without touching screens. Schema is standard Postgres so it dumps to any Postgres/SQLite via SQL/CSV export.

9. Database Requirement

We use a **cloud database (Supabase Postgres)**, which satisfies clause 9 (local *or* cloud). Auth + data live in Supabase; SharedPreferences caches only session/theme. Internet is required for sync; forms validate offline and show friendly errors. Because schema is plain Postgres + repository abstraction, we can migrate to SQLite/Hive/Firebase later with no UI rewrite and submit an offline fallback if needed.

10. Expected Database Structure

Three tables. One user has many expenses; one category has many expenses.

Table	Key fields (type)	Notes / relation
profiles	id (uuid PK = auth.uid), name (text), email (text), monthly_budget (numeric), updated_at	1 row per user; RLS: id = auth.uid()
categories	id (int PK), name (text), icon (text)	Seeded: Food, Transport, Hostel, Study, Health, Other

Table	Key fields (type)	Notes / relation
expenses	id (uuid PK), user_id (uuid FK), title (text), amount (numeric >0), type (income|expense), category_id (int FK), date (date), note (text), created_at	user_id→profiles.id; category_id→categories.id; RLS: user_id = auth.uid()

Table 2: Proposed schema. Indexes on (user_id, date) and (user_id, category_id) for filter performance.

Relationships: profiles 1—N expenses; categories 1—N expenses. Example row: expenses('...', user 'Zain', 'Mess bill', 2500, expense, Food, 2026-10-03). Budget check = SUM(expenses.amount WHERE type=expense AND month=current) vs profiles.monthly_budget.

12. Scope & Limitations

In scope: auth, expense CRUD, search/filter, monthly dashboard + charts, budget + local notification, profile, validation/error handling, UI + APK + functional testing per Assignments 2–4.

Out of scope (will not build): bank/UPI auto-sync, multi-currency, receipt OCR, friend splitting, admin web panel, offline-first two-way sync, iOS release. Internet required for cloud sync in v1; no SMS/permissions beyond notifications.

13. Future Enhancements

- **CSV export + full offline mode** (SQLite drift) for complete Supabase independence.
- **Recurring expenses + savings goals** with scheduled reminders.
- **Urdu/English toggle + dark mode** and monthly PDF report share.